MPC 분산 키 생성 프로토콜의 보안 모델 및 수학적 기초

개요

이 문서는 TNO PET Lab의 MPC 분산 키 생성 프로토콜의 보안 모델과 수학적 기초를 상세히 설명합니다. 프로토콜의 안전성이 어떻게 보장되는지와 사용된 암호학적 기법들을 다룹니다.

1. 보안 모델

1.1 반정직(Semi-Honest) 보안 모델

프로토콜은 **반정직 적대자 모델**을 가정합니다:

적대자의 행동 특성

- **프로토콜 준수**: 적대자는 정해진 프로토콜을 정확히 따릅니다
- **수동적 공격**: 통신 내용을 관찰하여 추가 정보를 얻으려 시도합니다
- **연합 공격**: 여러 악의적 참가자가 협력할 수 있습니다
- **적응적 부패**: 프로토콜 실행 중 참가자를 부패시킬 수 있습니다

보안 보장

```
정리 1 (반정직 보안성):

t < n/2인 경우, 최대 t명의 반정직 적대자가 존재해도

프로토콜은 다음을 만족합니다:

1. 정확성(Correctness): 정직한 참가자들은 올바른 키를 얻습니다
```

```
2. 프라이버시(Privacy): 적대자는 정직한 참가자의 비밀을 학습할 수 없습니다
```

1.2 임계값 보안

임계값 조건

프로토콜의 보안성을 위해 다음 조건이 필요합니다:

```
n ≥ 2t + 1
```

여기서:

- n: 전체 참가자 수
- t: 최대 악의적 참가자 수 (corruption threshold)

조건의 필요성

1. **비밀 분산**: t-out-of-n Shamir 비밀 분산 사용
2. **곱셈 연산**: 분산된 값들의 곱셈 시 차수가 2t로 증가
3. **재구성**: 2t차 다항식 재구성을 위해 2t+1개의 점이 필요

1.3 통계적 보안

통계적 거리 기반 보안

```
stat_sec_shamir = 40  # 통계적 보안 매개변수
```

```
# 마스킹 값의 범위
masking_range = 2^(prime_length + stat_sec_shamir)
```

```
# 통계적 거리

statistical_distance ≤ 2^(-stat_sec_shamir) = 2^(-40)
```

보안 보장

- 실제 분포와 균등 분포 간의 통계적 거리가 2^(-40) 이하
- 적대자가 마스킹된 값으로부터 원본 값을 구별할 확률이 무시할 수 있을 정도로 작음

2. Paillier 암호화 시스템의 수학적 기초

2.1 Paillier 암호화 스킴

키 생성

1. **소수 선택**: p, q를 큰 소수로 선택 (|p| = |q| = k/2 비트)
2. **모듈러스**: N = p × q
3. **카마이클 함수**: λ = lcm(p-1, q-1)
4. **생성자**: g = N + 1 (일반적인 선택)
5. **공개키**: (N, g)
6. **비밀키**: λ

암호화

평문 m ∈ Z_N에 대해:

```
E(m, r) = g^m × r^N mod N²
```

여기서 r ∈ Z_N*는 랜덤 값

복호화

암호문 c에 대해:

$$D(c) = L(c^\lambda \bmod N^2) \times \mu \bmod N$$

여기서:

- $L(x) = (x-1)/N$
- $\mu = (L(g^\lambda \bmod N^2))^{(-1)} \bmod N$

2.2 동형 암호화 성질

덧셈 동형성

$$E(m_1) \times E(m_2) = E(m_1 + m_2)$$

증명:

$$E(m_1) \times E(m_2) = g^{m_1} \times r_1^N \times g^{m_2} \times r_2^N \bmod N^2$$

$$= g^{(m_1 + m_2)} \times (r_1 \times r_2)^N \bmod N^2$$

$$= E(m_1 + m_2, r_1 \times r_2)$$

스칼라 곱셈

$$E(m)^k = E(k \times m)$$

2.3 분산 복호화의 수학적 원리

비밀키 분산

비밀키 λ를 Shamir 비밀 분산으로 분산:

$$\lambda = \Sigma_i \lambda_i \times L_i(0)$$

여기서 $L_i(x)$는 라그랑주 보간 다항식

부분 복호화

참가자 i의 부분 복호화:

$$c_i = c^{(\Delta \times \lambda_i)} \bmod N^2$$

여기서 Δ = n! (팩토리얼)

최종 복호화

$$c^{\lambda} = \Pi_i c_i^{(L_i(0)/\Delta)} \bmod N^2$$

라그랑주 보간에 의해:

$$\Pi_i c_i^{(L_i(0)/\Delta)} = \Pi_i (c^{(\Delta \times \lambda_i)})^{(L_i(0)/\Delta)}$$

$$= \Pi_i c^{(\lambda_i \times L_i(0))}$$

$$= c^{(\Sigma_i \lambda_i \times L_i(0))}$$

$$= c^{\lambda}$$

3. Shamir 비밀 분산의 수학적 기초

3.1 기본 원리

다항식 기반 비밀 분산

비밀 s를 분산하기 위해 t차 다항식 f(x) 구성:

$$f(x) = s + a_1 x + a_2 x^2 + \dots + a_t x^t \bmod p$$

여기서:

- s: 분산할 비밀
- $a_1, \dots, a_t$: 랜덤 계수
- p: 큰 소수

분산 생성

각 참가자 i에게 분산 (i, f(i)) 제공

비밀 재구성

t+1개의 분산을 사용하여 라그랑주 보간으로 f(0) = s 계산:

$$s = \Sigma_i \, y_i \times L_i(0)$$

여기서:

$$L_i(0) = \Pi_{j \neq i} \, (-j)/(i-j)$$

3.2 분산 연산

덧셈

두 비밀 a, b가 분산되어 있을 때:

```
[a] + [b] = [a + b]
```

각 참가자가 자신의 분산을 더하면 됨:

```
(i, fₐ(i) + f_b(i)) = (i, f_{a+b}(i))
```

곱셈

```
[a] × [b] = [a × b]
```

하지만 결과의 차수가 2t가 되므로 2t+1개의 분산이 재구성에 필요

3.3 정수 상의 Shamir 비밀 분산

문제점

일반적인 Shamir 비밀 분산은 유한체에서 동작하지만,

Paillier 암호화에서는 정수 연산이 필요

해결책: 통계적 보안

```
# 마스킹 범위 확장
masking_range = 2^(bit_length + statistical_security_parameter)

# 분산 생성 시 큰 범위에서 랜덤 계수 선택
coefficients = [random.randrange(masking_range) for _ in  range(threshold)]
```

보안성 분석

- 원본 비밀의 크기: 2^k
- 마스킹 범위: 2^(k + σ)
- 통계적 거리: ≤ 2^(-σ)

4. 분산 소수성 검사

4.1 Miller-Rabin 테스트의 분산 버전

기본 Miller-Rabin 테스트

소수 후보 n에 대해:

1. n-1 = 2^r × d (d는 홀수)
2. 랜덤 a ∈ [2, n-2] 선택
3. x = a^d mod n 계산
4. x = 1 또는 x = n-1이면 통과
5. r-1번 반복: x = x^2 mod n, x = n-1이면 통과

분산 버전

각 참가자가 독립적으로 테스트를 수행하고 결과를 교환:

```
async def distributed_primality_test(candidate, generators, index, pool, party

# 각 참가자가 여러 생성자로 테스트

test_results = []

for g in generators:

# Miller-Rabin 테스트 수행

result = miller_rabin_test(candidate, g)
```

```
test_results.append(result)

# 결과를 다른 참가자들과 교환

pool.async_broadcast(test_results, msg_id=f"primality_test_{session_id}_{candida

responses = await pool.recv_all(msg_id=f"primality_test_{session_id}_{candidate}

# 모든 참가자의 결과가 일치하는지 확인

for _, other_results in responses:

if other_results != test_results:

return False

return  all(test_results)
```

4.2 이중 소수 검증

목적

생성된 N = p × q가 정확히 두 소수의 곱인지 확인

방법: 이차잉여 판정

소수 p에 대해 생성자 g의 이차잉여성 확인:

```
g^((p-1)/2) ≡ ±1 (mod p)
```

N = p × q가 두 소수의 곱이면:

```
g^((N-1)/2) ≡ g^((p-1)/2) × g^((q-1)/2) (mod N)
```

분산 구현

```
async def distributed_biprimality_test(n, generators, index, pool, party_indic

test_results = []

for g in generators:

# g^((n-1)/2) mod n 계산

exponent = (n - 1) // 2

result = pow(g, exponent, n)

# 결과가 ±1인지 확인

test_results.append(result in [1, n-1])

# 결과 교환 및 검증

# ... (이전과 동일한 패턴)

return all(test_results)
```

5. 보안성 증명 스케치

5.1 시뮬레이터 기반 증명

정리 2 (보안성):

```
프로토콜 Π가 이상적 기능 F를 안전하게 구현한다는 것은

모든 실제 적대자 A에 대해 시뮬레이터 S가 존재하여:

{REAL_Π,A(x)}_{x∈X} ≈ {IDEAL_F,S(x)}_{x∈X}
```

시뮬레이터 구성

1. **입력 단계**: 적대자의 입력을 받아 이상적 기능에 전달
2. **중간 메시지**: 적대자가 볼 수 있는 메시지들을 시뮬레이션
3. **출력 단계**: 이상적 기능의 출력을 적대자에게 전달

5.2 프라이버시 보장

정보 이론적 보안

Shamir 비밀 분산의 완벽한 비밀성:

```
정리 3: t개 이하의 분산으로는 비밀에 대한 정보를 전혀 얻을 수 없다.
```

증명: t개의 분산은 t차 다항식의 t개 점이므로,

상수항(비밀)에 대해 모든 값이 동일한 확률을 가짐.

통계적 보안

정수 상의 비밀 분산에서:

```
|Pr[A outputs 1 | real] - Pr[A outputs 1 | simulated]| ≤ 2^(-σ)
```

5.3 강건성(Robustness)

정의

악의적 참가자가 있어도 정직한 참가자들은 올바른 결과를 얻을 수 있음

보장 메커니즘

1. **일관성 검사**: 모든 중간 결과에 대한 검증
2. **재시도 메커니즘**: 실패 시 새로운 후보로 재시작

3. **임계값 조건**: $n \geq 2t + 1$ 조건 강제

6. 성능 분석

6.1 통신 복잡도

라운드 복잡도

- **인덱스 할당**: O(1) 라운드
- **소수 생성**: O(k) 라운드 (k는 후보 수)
- **모듈러스 계산**: O(1) 라운드
- **복호화**: O(1) 라운드

메시지 복잡도

- **총 메시지 수**: $O(n^2)$
- **메시지 크기**: O(|N|) = O(키 길이)

6.2 계산 복잡도

소수 생성

- **Miller-Rabin 테스트**: $O(\log^3 n)$ per test
- **분산 검증**: $O(n \times \log^3 n)$

분산 연산

- **Shamir 분산 생성**: $O(n^2)$
- **분산 곱셈**: $O(n^2)$
- **재구성**: $O(n^2)$

6.3 최적화 기법

배치 처리

```
# 여러 후보를 동시에 처리하여 통신 라운드 감소
batch_size = 100
candidates = generate_candidate_batch(batch_size)
valid_candidates = await distributed_batch_primality_test(candidates)
```

사전 계산

```
# 작은 소수들과 생성자들을 미리 계산
small_primes = list(sympy.primerange(3, prime_threshold))
generators = generate_quadratic_residues(correct_param_biprime * JACOBI_CORRECTI
```

이 문서는 MPC 분산 키 생성 프로토콜의 보안성이 어떻게 수학적으로 보장되는지를 상세히 설명하여, 프로토콜의 이론적 기초를 완전히 이해할 수 있도록 도와줍니다.